

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO

Relatório Experimental - Projeto 1: Busca e Organização de Dados

Disciplina: Algoritmos e Estruturas de Dados I (SCC0202)

Carolina Goulart Kowalczuk, N°USP: 13854629
Júlia Barbosa Nogueira, N°USP: 17901347
Victor Hugo Adão de Oliveira, N°USP: 17969072

1. Tabelas da Avaliação Experimental

Tempos exibidos em milissegundos, sendo a média de 5 execuções.
Última linha: base completa (89.227 registros), exportando todos os registros encontrados (3.497).
Consulta: q1_asteroides.

1.1 pilha + pilha

N	Base	Resultado	k	Achados	Carga	Busca	Inserções.	Saída	Total
1.000	pilha	pilha	10	68	5,41	0,28	0,01	0,19	5,89
5.000	pilha	pilha	50	332	24,85	1,36	0,04	0,23	26,48
10.000	pilha	pilha	100	615	47,34	2,64	0,08	0,26	50,31
25.000	pilha	pilha	500	1.395	119,23	7,13	0,21	0,61	127,18
50.000	pilha	pilha	1.000	2.091	222,65	10,10	0,30	3,05	236,10
89.227 (toda)	pilha	pilha	todos (3.497)	3.497	387,54	16,84	0,42	2,44	407,24

1.2 fila + fila

N	Base	Resultado	k	Achados	Carga	Busca	Inserções.	Saída	Total
1.000	fila	fila	10	68	4,72	0,26	0,01	0,19	5,17
5.000	fila	fila	50	332	25,64	1,42	0,05	0,23	27,32
10.000	fila	fila	100	615	43,85	2,51	0,08	0,25	46,68
25.000	fila	fila	500	1.395	105,52	5,85	0,17	0,53	112,07
50.000	fila	fila	1.000	2.091	210,46	10,03	0,27	0,86	221,62
89.227 (toda)	fila	fila	todos (3.497)	3.497	375,30	16,89	0,42	4,73	397,34

1.3 lista_seq + lista_seq

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_seq	lista_seq	10	68	4,93	0,25	0,01	0,20	5,37
5.000	lista_seq	lista_seq	50	332	21,81	1,13	0,03	0,19	23,17
10.000	lista_seq	lista_seq	100	615	43,21	2,22	0,06	0,24	45,72
25.000	lista_seq	lista_seq	500	1.395	104,30	5,39	0,13	0,51	110,33

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
50.000	lista_seq	lista_seq	1.000	2.091	208,77	8,80	0,16	0,83	218,56
89.227 (toda)	lista_seq	lista_seq	todos (3.497)	3.497	374,03	15,03	0,28	4,68	394,02

1.4 lista_enc + lista_enc

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_enc	lista_enc	10	68	5,76	0,30	0,01	0,21	6,28
5.000	lista_enc	lista_enc	50	332	22,52	1,24	0,05	0,20	24,02
10.000	lista_enc	lista_enc	100	615	43,22	2,43	0,08	0,27	45,99
25.000	lista_enc	lista_enc	500	1.395	105,56	5,75	0,17	0,52	112,00
50.000	lista_enc	lista_enc	1.000	2.091	212,22	9,84	0,28	0,86	223,20
89.227 (toda)	lista_enc	lista_enc	todos (3.497)	3.497	385,32	18,05	0,52	2,46	406,35

1.5 lista_cab + lista_cab

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_cab	lista_cab	10	68	4,78	0,26	0,01	0,23	5,28
5.000	lista_cab	lista_cab	50	332	22,62	1,29	0,04	0,20	24,15
10.000	lista_cab	lista_cab	100	615	43,29	2,44	0,08	0,23	46,04
25.000	lista_cab	lista_cab	500	1.395	105,71	5,86	0,16	0,52	112,25
50.000	lista_cab	lista_cab	1.000	2.091	209,85	9,81	0,25	0,84	220,75
89.227 (toda)	lista_cab	lista_cab	todos (3.497)	3.497	378,56	16,74	0,42	2,44	398,16

1.6 lista_ord + lista_ord

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_ord	lista_ord	10	68	5,53	0,25	0,05	0,20	6,02
5.000	lista_ord	lista_ord	50	332	26,01	1,37	0,28	0,23	27,89

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
10.000	lista_ord	lista_ord	100	615	53,13	2,70	0,57	0,25	56,65
25.000	lista_ord	lista_ord	500	1.395	151,67	7,27	1,70	0,54	161,19
50.000	lista_ord	lista_ord	1.000	2.091	441,16	13,05	2,86	0,92	457,99
89.227 (toda)	lista_ord	lista_ord	todos (3.497)	3.497	1.154,17	23,42	5,90	5,22	1.188,71

1.7 lista_gen + lista_gen

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_gen	lista_gen	10	68	5,97	0,28	0,07	0,19	6,51
5.000	lista_gen	lista_gen	50	332	27,27	1,42	0,33	0,21	29,22
10.000	lista_gen	lista_gen	100	615	53,89	2,90	0,60	0,26	57,66
25.000	lista_gen	lista_gen	500	1.395	132,59	7,56	1,46	0,56	142,17
50.000	lista_gen	lista_gen	1.000	2.091	269,40	13,02	2,30	0,92	285,65
89.227 (toda)	lista_gen	lista_gen	todos (3.497)	3.497	501,87	23,78	4,57	4,93	535,15

Observação 1: a **lista_gen** teve carga cerca de 1,2 a 1,3 vez maior que a das listas simples (por exemplo, 132,59 ms contra 104,30 ms da **lista_seq** em N = 25.000, e 501,87 ms contra 374,03 ms na base completa). Cada inserção desce pela hierarquia de grupos: localiza a sublista da categoria (**threat_category**) por busca linear, depois a sublista do ano (**year**) dentro dela, e só então anexa o item no fim do subgrupo (O(1), pois cada nível guarda ponteiro para o fim). O custo por inserção é O(L + A), sendo L o número de categorias e A o número máximo de anos distintos dentro de uma categoria, e o da carga completa é O(N · (L + A)). Como L e A são pequenos e limitados, o crescimento observado é praticamente linear em N, com constante maior. Na inserção do resultado o efeito é mais visível (4,57 ms contra 0,28 a 0,52 ms nas listas simples, com 3.497 achados), porque ali não há leitura de arquivo diluindo o custo da busca do grupo.

1.8 lista_cruz + lista_cruz

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
1.000	lista_cruz	lista_cruz	10	68	5,86	0,28	0,07	0,20	6,41
5.000	lista_cruz	lista_cruz	50	332	27,50	1,44	0,32	0,21	29,47
10.000	lista_cruz	lista_cruz	100	615	75,77	3,65	0,71	0,30	80,43

N	Base	Resultado	k	Achados	Carga	Busca	Inserção res.	Saída	Total
25.000	lista_cruz	lista_cruz	500	1.395	168,95	8,93	1,59	0,59	180,05
50.000	lista_cruz	lista_cruz	1.000	2.091	314,23	15,49	2,55	3,16	335,43
89.227 (toda)	lista_cruz	lista_cruz	todos (3.497)	3.497	562,57	28,58	5,44	3,13	599,72

Observação 2: a **lista_cruz** teve carga cerca de 1,3 a 1,6 vez maior que a das listas simples (por exemplo, 168,95 ms contra 104,30 ms da **lista_seq** em N = 25.000). Cada inserção precisa localizar o grupo da linha (**threat_category**) e o da coluna (**year**) por busca linear na lista de grupos, com comparação de strings, e só então encadear o nó no fim de cada um (O(1), pois há ponteiro para o fim). O custo por inserção é, portanto, O(L + C), sendo L o número de categorias e C o número de anos, e o da carga completa é O(N · (L + C)). Como L e C são pequenos e limitados, o crescimento observado é praticamente linear em N, mas com constante maior. Em N = 10.000, o valor medido (75,77 ms) ficou acima da tendência; uma nova medição isolada resultou em cerca de 62 ms, indicando que parte dessa diferença foi variação do sistema.

Observação 3: Sobre essas duas últimas estruturas, é importante perceber que em ambas, o custo por inserção depende do número de grupos (categorias e anos), e não apenas de N: como esse número é pequeno e praticamente fixo no dataset, a carga cresce de forma linear em N, mas, se o número de grupos aumentasse junto com a base, a busca linear do grupo tornaria a inserção mais cara e o crescimento deixaria de ser linear.
Além disso, é perceptível que a lista_gen ficou mais rápida que a lista_cruz (269,4 contra 314,2 ms em N = 50.000 de acordo com as tabelas de análise). Isso ocorre porque na lista cruzada, a busca da coluna percorre todos os anos existentes. Na generalizada, o ano é procurado só dentro da categoria já encontrada, que tem menos grupos. A cruzada cria e liga dois encadeamentos por nó.

2. Análise da complexidade assintótica:

Legenda: N: número de elementos.
L: número de categorias (threat_category).
A: máximo de anos distintos dentro de uma categoria.
C: número de anos distintos.
G: total de sublistas.

Estrutura	Inserção	Busca	Remoção	Observações
pilha	O(1)	O(N)	O(1) (topo)	Busca é o percurso completo da estrutura.
fila	O(1)	O(N)	O(1) (início)	Busca é o percurso completo da estrutura.
lista_seq	O(1)	O(N)	O(N)	Inserção no fim; remoção exige deslocamento.

Estrutura	Inserção	Busca	Remoção	Observações
lista_enc	O(1)	O(N)	O(N)	Inserção no fim com ponteiro para o fim; remoção exige localizar o nó.
lista_cab	O(1)	O(N)	O(N)	O nó sentinela evita casos especiais, sem mudar a ordem de grandeza.
lista_ord	O(N) por inserção; $O(N^2)$ para N itens	O(N)	O(N)	A ordem é mantida durante a inserção.
lista_gen	O(L + A)	O(N + G)	não utilizada	Busca linear do grupo e do subgrupo; item anexado no fim em O(1). O código não tem remoção de item individual, apenas lgen_destruir, que é O(N+G)
lista_cruz	O(L + C)	O(N + L)	não utilizada	Um único nó por item, ligado à lista de linha e à de coluna.

2.1 Comparação com as medições (complexidade esperada vs. tempos medidos):

Na <u>Carga:</u>

As estruturas com inserção O(1) (pilha, fila, lista sequencial, lista encadeada e lista com cabeça) apresentaram o crescimento linear esperado: na lista sequencial, o tempo passou de 104,30 ms para 208,77 ms quando N foi de 25.000 para 50.000 (razão 2,0) e chegou a 374,03 ms com 89.227 registros (razão 1,8 para um N 1,78 vez maior), mantendo ≈4,2 µs por registro, pois o custo da carga (que inclui a leitura e a interpretação do CSV) é igual para qualquer estrutura.

A lista ordenada, esperada como $O(N^2)$, confirmou o crescimento quadrático: descontada a carga da lista sequencial, o custo extra passou de 47,4 ms para 232,4 ms e 780,1 ms, com expoente entre 2,1 e 2,3. OBS.: essa lista é a única estrutura que foge do comportamento linear. Ademais, é importante ressalvar que, na prática, o termo quadrático só domina em N grande, porque a constante de cada deslocamento ou comparação é pequena perto do custo de interpretar uma linha do CSV.

As listas generalizada e cruzada, esperadas como $O(N\cdot(L+A))$ e $O(N\cdot(L+C))$, ficaram em cerca de 1,3 e 1,5 a 1,6 vez o tempo da lista sequencial, com crescimento aproximadamente linear, o que é coerente com um número pequeno e fixo de grupos. A cruzada custa mais porque percorre todos os anos existentes (C), enquanto a generalizada procura o ano apenas dentro da categoria já encontrada (A), e porque a cruzada liga cada nó a dois encadeamentos.

Na <u>Busca:</u>

Todas as estruturas percorrem a base inteira e foram esperadas como O(N). Os tempos confirmam isso: de 0,28 ms com N = 1.000 a 15 a 29 ms com N = 89.227, ou de 0,17 a 0,32 µs por registro. A diferença entre as estruturas está na constante e não na ordem de grandeza: pilha, fila e listas simples percorrem os nós na mesma ordem em que foram alocados, o que favorece o uso do cache. Nas listas ordenada, generalizada e cruzada, o percurso segue a ordem das chaves ou dos grupos, que não coincide com a

ordem de alocação, e isso explica o tempo maior (23 a 29 ms contra 15 a 18 ms). Em N pequeno, o tempo é muito curto e o ruído pesa mais, por isso a razão entre tamanhos consecutivos não é exatamente 2.

Na <u>Inserção no resultado:</u>

As estruturas simples, esperadas como O(1) por item, gastaram entre 0,28 e 0,52 ms para 3.497 achados, cerca de 0,1 µs por item.

A lista ordenada, esperada como O(k²) no total, cresceu de forma superlinear, mas abaixo do quadrático: com 51 vezes mais achados (de 68 para 3.497), o tempo cresceu 125 vezes, com expoente próximo de 1,2.

OBS.: Note que o comportamento quadrático aparece com clareza apenas na carga (89.227 itens), e não nos 3.497 itens do resultado, pois, como o custo por inserção é O(k) com constante muito pequena e como a posição de inserção depende da ordem em que os registros chegam: se já chegam parcialmente ordenados, cada inserção desloca ou percorre menos elementos que no pior caso.

As listas generalizada e cruzada gastaram 4,57 e 5,44 ms, de 10 a 13 vezes o das estruturas simples, o que reflete a busca linear do grupo a cada inserção. Na saída, o tempo é praticamente constante para k pequeno (≈0,2 ms, custo de abrir e fechar o arquivo) e cresce linearmente com k, a ≈0,6 a 0,7 µs por registro, como esperado para O(k).

Na <u>Saída:</u>

O tempo de saída depende de k, e não de N. Com k = 10 é praticamente constante, em torno de 0,2 ms, que corresponde ao custo fixo de abrir o arquivo, escrever o cabeçalho e fechá-lo. Ao exportar milhares de registros, o tempo cresce de forma linear: ≈0,6 a 0,7 µs por registro, 2,4 a 5,2 ms para os 3.497 registros. Isso confirma O(k).

Algumas medições isoladas (por exemplo, 3,05 ms na pilha com N = 50.000) destoam por ruído do sistema, já que os valores são frações de milissegundo.

2.2 Conclusões:

No geral, os tempos medidos confirmam as complexidades teóricas: as operações de carga e busca são lineares, com exceção da carga na lista ordenada, que é quadrática. Nas estruturas simples, a escolha do tipo pouco impacta no desempenho, pois a inserção é imediata e o tempo total é dominado pela leitura do arquivo.

Em contrapartida, o custo superior das estruturas avançadas reflete o preço de manter a organização dos dados (seja por ordenação, hierarquia de grupos ou múltiplas visões por referências cruzadas). Portanto, embora a lista ordenada seja ineficiente para carregar grandes volumes, ela se mostra viável para armazenar conjuntos pequenos de resultados.

2.2.1 Limitações e observações finais:

Os tempos de inserção e de saída em N pequeno são frações de milissegundo e sofrem mais com o ruído do sistema. A carga mede juntas a leitura do CSV e a inserção na base.

A medição da inserção no resultado consulta o relógio do sistema duas vezes a cada item (antes e depois da inserção). Como cada consulta também leva algum tempo, os valores de inserção no resultado ficam levemente superestimados, principalmente nas estruturas O(1), cuja inserção é tão rápida quanto a própria medição.

Foram testadas apenas combinações com a mesma estrutura na base e no resultado, e a variação entre execuções (de até ≈4% nas cargas O(1)) limita a precisão de comparações entre estruturas parecidas.

Em uma bateria anterior, executada com outros programas abertos e com uma média de apenas 3 execuções, os tempos ficaram claramente mais ruidosos. Os dados apresentados vêm da execução com o computador sem outras cargas.